`failWith(error: Throwable)` and have the `ExceptionHandler` handle it.

Whether exceptions are thrown by you intentionally or by an unexpected circumstance, the sooner you can catch them, the better. Exceptions have significant performance penalties every time they are handled. This is a JVM feature, not an Akka one. A significant cost is associated with creating a stack trace for each nested `try`/`catch` and unwinding all the levels when a new exception is thrown.

Let's look at an example of adding an exception handler to prevent an `ArithmeticException` from bubbling up to the default exception handler. This route fails quite often because it randomly divides a random number by 0 or 1. Half the time, it ends up dividing by zero and issuing a message that something went wrong with the URI. To extract the URI, the directive `extractUri` is used. Other directives like this—for example, `extractEntity`—are often used in exception handlers to extract information from the request.

Listing 11.2 Adding an `ExceptionHandler` to a route

```
object HttpServerExceptions {
  ...
    def exceptionHandler = ExceptionHandler {      ❶
      case _: ArithmeticException =>
        extractUri { uri =>                        ❷
          complete(
            HttpEntity(
              ContentTypes.`application/json`,
              s"sorry something went wrong with $uri")
          )
        }
    }

    val route: Route =
```